

Pipes ([Old Notes](#))

Introduction

- So far we have learned one way for two processes to communicate. What is it? Signals!?!? That is using a signal to nudge another process.
- Pipe system call is used to facilitate one way communications between two processes. They allow us to pass information.
- The communications goes through a buffer (i.e. queue) accessible by the operating system.
- Processes can read from and write to the pipe through file descriptors. (In the same way you do to a file.)
- **Note: This material, besides useful in its own right, is much more important in that it will help in the learning of the process-to-process communications methods and ideas behind TCP/IP communications.**

Creating a pipe Review starting here on 10/22/2025

The following is the prototype for the pipe system call.

```
#include <unistd.h>
#include <limits.h>
int pipe( int fildes[2] );
fildes[0] is the read end of the pipe and fildes[1] is the write end of the pipe.
```

Note: if two processes are to communicate over pipes, they must be related. Why? - Otherwise they would not have access to the pipe's file descriptors. Recall that a child process inherit open file descriptors from its parent.

Note: The size of a pipe on Linux is typically 65536 bytes. One can set the size of pipes programmatically. (We will do this later.) Using `ulimit -p` I got 4K. Something is wrong!

See example [pipe1.cpp](#). **Must emphasize that pipes are stream oriented. If you send a series of messages there can be problems. It is your job to break the stream of data into the original messages.** This is a similar issue to communication over the Internet.

Notes on example:

1. Will show the basic mechanics of the call.
2. Will show that the call is stream oriented, by pausing the child for 10 seconds.
3. Will show a zero length read if there are no open "write" file descriptors.
4. Will show that the write will fail if there are no open read file descriptors.
5. Will show that the write will eventually block, if no one reads. We can use this to deduce the size of pipes. (ulimit -p will also do the job. Maybe not so well) Let's try to get the size of a pipe.
6. **A general question. Why should you get rid of compiler warnings in your code?**

The fcntl system call

fcntl is a system call that may be used to change the characteristics of a file descriptor.

```
#include <fcntl.h>
```

```
int fcntl (int fd, int request, int arg);
```

fd - is a file descriptor.

request - indicates how we wish to change the characteristics of the file descriptor.

arg - an optional argument.

You can use this function for many purposes. I will not give all the details.

We will use fcntl to make a file descriptor non-blocking - see example [pipe2.cpp](#).

Note how we first use an option to get the file flags, so we can add

O_NONBLOCK to the set.

We can also get (F_GETPIPE_SZ option) and set (F_SETPIPE_SZ option) the size of a pipe. The set can go up to the limit set for the system.

Note If we make a pipe non-blocking, then errno is EAGAIN if the read call would have blocked. This is true even though the description of the error given by perror is: "Resource temporarily unavailable". Note that we can now tell the size of pipes in the operating system. Ours uses 64K. (Last tested 10/14/2025) How would we test? How would we find out without testing.

dup2 is a system call that allows you to copy the data associated with one file descriptor into another.

```
int dup2( int oldfd, int newfd);
```

If the newfd is open file descriptor, it is closed before any copying is done.

dup2 may be used to do simulate the piping done by the shells. That is we can redirect output from the screen to a file.

Note: there are other ways to duplicate a fd, but this method is the simplest and cleanest. (See the dup call for another way i.)

See example on redirection. [pipe3.cpp](#) It also shows how a process run using an exec call can inherit an open file descriptor. We use the sort command. I will demonstrate how the command works and how we can use it in a program. **"^d" means we are done typing for the sort.**

Lab8

Write a program that will do the following:

1. It will create two pipes.
2. Create a clone of itself.
3. Prompt the user for a string.
4. Send the string over one pipe to the child process.
5. The child process will reverse the characters in the string and send them back to the parent over the second pipe.
6. The parent will display the string that was sent to it.

Select

- Select system call. It provides an effective way of dealing with the possibility of data coming in from a number of pipes.
- It is also useful in a number of other situations include **TCP/IP coding. Picture a server with 100 connections on what look like two way pipes.** How do we deal with this?
- **This is a difficult call to master.** But, it is an important call for other work (E.g. network programming) Please pay particular attention to the discussion in class.

```
#include <sys/types.h>
#include <sys/time.h>
```

```
int select (int nfds, fd_set *readfds, fd_set *writefds, fd_set *exceptfds
            , struct timeval *timeout);
```

nfds - maximum value file descriptor + 1.

readfds - set of file descriptors for which there is data ready to read. In Linux it is a bit map.

writelfds - set of file descriptors that we are waiting to be ready to be written to. That means that there is at least one byte that may be written.

exceptfds - set of file descriptors for which we are waiting for exception. **Don't worry about this one for now.**

timeout - specifies how long this call should wait if there is not data available.

NULL for this argument means wait forever. 0 for the time values means no wait.

```
struct timeval {
    long    tv_sec;        /* seconds */
    long    tv_usec;       /* and microseconds */
};
```

Returns a count of the number of ready file descriptors. The fd_set variables are set to indicate which file descriptors are ready. So, readfds will indicate the file descriptors ready to be read.

The following macros/functions are used to manipulate the bit maps.

```
FD_SET (int fd, fd_set *fdset)
FD_CLR (int fd, fd_set *fdset)
FD_ISSET (int fd, fd_set *fdset)
FD_ZERO (fd_set *fdset)
```

Note: at one time select was used as a more precise sleep function that does not use SIGALRM. **Linux has since developed better sleep functions, so this use has become irrelevant - or should be - you might see it used in older code.** (See **usleep** and **nanosleep**)

Note: Microsoft does offer a select call. It ignores the nfds parameter. **It is important to know this - Why?** Note: Microsoft's fd_set has a different internal format.

See examples. [select1.cpp](#) [select2.cpp](#) Notes

- select1 is an example of how to use select as a more precise sleep. This is no longer needed. How do we find a different way?
- The select2 example uses only one pipe. Clearly select is much more useful with more than one pipe. See the next lab.

Note: there is also the poll and epoll system calls that do approximately the same thing. We don't have time to discuss. They just allow a little more flexibility and efficiency. A short discussion of epoll follows:

The epoll call.

Notes on select, poll, and epoll. We will discuss later.

- select is the oldest call. It has the advantage of being portable across various Unix and Unix-like systems.
- The select call has a limit of 1023 file descriptors. This is due to the fd_set type. So, we need another call if there might be more file descriptors to monitor.
- The epoll call can support any number of file descriptors. There is a performance issue, with select, in that on each call you have to scan all the file descriptors and so does the system. This can lead to a speed difference of more than 100 for handling large sets of file descriptors.
- The epoll call is specific to Linux. It using the operating system to tell it which file descriptors have information pending. It is vastly more efficient for large sets of file descriptors. So it should be your choice if you are running a large application.
- The basic calls used with epoll are:
 - epoll_create or epoll_create1 - creates an epoll instance. This is used by Linux to identify a set of file descriptors that will be monitored.
 - epoll_ctl - used to specify the file descriptors to be monitored and what the will be monitored for. For example ready to be read. Note: the primary reason that epoll is much faster than select is that the file descriptors only have to be set up once with epoll, where they are set up with each call to select.
 - epoll_wait - used to wait until at least one of the file descriptors are ready. When it returned it will supply a list of the file descriptors that are "ready".

epoll_create and epoll_create1

```
int epoll_create( int size );
```

I will not discuss this system call. It is obsolete. But, in case you need it, it will return file descriptor for an epoll instance. The size argument is ignored. It must be greater than zero. The reason I mention it is that there is a lot of legacy code and I don't want you to get surprised. As with most calls, a return value of -1 indicates an error condition.

```
int epoll_create1( int flags );
```

Returns a file descriptor referencing a new epoll instance. The epoll instance will be used to track the state of a user defined set of file descriptors.

flags - currently there is only one flag: FD_CLOEXEC. This indicates that if there is an exec call, the file descriptor for epoll will not be open in the child process. Note: this is also an option for the open call.

epoll_ctl This allows us to add, modify and delete entries from the interest list associated with the epoll file descriptor.

```
int epoll_ctl(int epfd, int op, int fd, struct epoll_event *event);
```

epfd - a file descriptor created by epoll. This references the epoll instance.

fd - the file descriptor to be monitored by epoll. (The operations *op* This is often referred to as the target file descriptor.)

op - the operation to be performed. The valid values are below.

EPOLL_CTL_ADD

Register (i.e. add) the target file descriptor *fd* on the **epoll** instance referred to by the file descriptor *epfd* and associate the event *event* with the internal file linked to *fd*.

EPOLL_CTL_MOD

Change the event *event* associated with the target file descriptor *fd*.

EPOLL_CTL_DEL

Remove (deregister) the target file descriptor *fd* from the **epoll** instance referred to by *epfd*. The *event* is ignored and can be NULL.

event - describes the event we are waiting for on the file descriptor *fd*. The format of an event is a little complicated. It is a pointer to structure `epoll_event`. See below.

```
typedef union epoll_data {
    void      *ptr;
    int       fd;
    uint32_t   u32;
    uint64_t   u64;
} epoll_data_t;

struct epoll_event {
    uint32_t   events;      /* Epoll events */
    epoll_data_t data;      /* User data variable */
}
```

```
};
```

The events member is a bit map of events to test. I have taken this information from the link: https://linux.die.net/man/2/epoll_ctl. I will not discuss all of them..

EPOLLIN

The associated file is available for **read**(2) operations.

EPOLLOUT

The associated file is available for **write**(2) operations.

EPOLLRDHUP (since Linux 2.6.17)

Stream socket peer closed connection, or shut down writing half of connection. (This flag is especially useful for writing simple code to detect peer shutdown when using Edge Triggered monitoring.)

EPOLLPRI

There is urgent data available for **read**(2) operations.

EPOLLERR

Error condition happened on the associated file descriptor. **epoll_wait**(2) will always wait for this event; it is not necessary to set it in *events*.

EPOLLHUP

Hang up happened on the associated file descriptor. **epoll_wait**(2) will always wait for this event; it is not necessary to set it in *events*.

EPOLLET

Sets the Edge Triggered behavior for the associated file descriptor. The default behavior for **epoll** is Level Triggered. See **epoll**(7) for more detailed information about Edge and Level Triggered event distribution architectures.

EPOLLONESHOT (since Linux 2.6.2)

Sets the one-shot behavior for the associated file descriptor. This means that after an event is pulled out with **epoll_wait**(2) the associated file descriptor is internally disabled and no other events will be reported by the **epoll** interface. The user must call **epoll_ctl**() with **EPOLL_CTL_MOD** to rearm the file descriptor with a new event mask.

The `epoll_data` element specifies the data that the kernel will save and return when it reports that an event has occurred. For us, it will usually be the file descriptor we are waiting on.

Return value

Returns 0 if successful and -1 if error conditions. See: https://man7.org/linux/man-pages/man2/epoll_ctl.2.html for a list of error conditions.

epoll_wait

Waits for events specified by the `epoll_ctl` systems call and reports any events that have occurred. There are variations of this call that will not be discussed here. Its prototype is:

```
int epoll_wait(int epfd, struct epoll_event events[.maxevents], int maxevents,  
int timeout);
```

`epoll` - the file descriptor that is an `epoll` instances. That is was created by: `epoll_create` or `epoll_create1`.

`maxevents` - the maximum number of events that will be reported. If there are more events, they will reported the next time that `epoll_wait` is called.

`events` - points to an array of `epoll_events`. (See `epoll_ctl` for the format of `epool_event`.) This array should be the size of `maxevents`. The events return a list of events that have occurred. The `fd` indicates the file descriptor and the `data` part is what was passed when `epoll_ctl` was called and `events` indicate the actual event that has occurred.

`timeout` - the number of milliseconds that the call will wait. 0 for the value means no wait. -1 means infinite timeout.

A call to `epoll_wait` will block until one of the following things occurs:

- An event occurs on one or more file descriptors.
- The timeout occurs.
- A signal occurs. In this case the function returns a -1 and `errno == EAGAIN`.

Returns: the number of events that were recorded in the `events` array. 0 is returned if there is a timeout and -1 if there is an error.

The following is a simple example example of using `epoll`. [EpollExample.cpp](#) Please note the following:

- The example will be waiting for data pending on standard input. This is the input would normally be read using `cin`. I will use the `read` function when there is data pending.
- The first thing we did was create an `epoll` file descriptor using `epoll_create1`. The OS will use this to identify the set of file descriptors for which we want information.

- The `epoll_ctl` function was next called to tell the OS to associated the file descriptor 0 with the `epoll` we created. I have to pass data that will be returned when the file descriptor has data pending. I used the constant `EPOLLIN` to indicate that I want the know when the file descriptor is read to be read.
- Finally, I have a loop where the function `epoll_wait` is called. This will cause our program to wait for any event that had previously be associated with the `epoll` file descriptor.
- Testing - let's try a few things:
 - Let a timeout occur and see how the program reacts.
 - type characters, but don't hit enter.
 - type more than 10 characters and hit enter.
 - Create a few pipes and clean up if they terminate. (If there is time)
 - send a signal. (If there is time)

The `mkfifo` call

- FIFO or named pipes are used when there is no family relationship between processes.
- A file name is used to identify the pipe. **Be careful with this. We will discuss in class.**
- The system call `mkfifo` is used to create this pipe.
- The pipe looks like a file. (A neat way to share a name.) Note: there is also a `mkfifo` command.

```
#include <sys/types>
#include <sys/stat.h>
int mkfifo(const char *path, mode_t mode );
```

`path` id the name of the file
`mode` is the permissions. These are standard file access permission. The latter is used when the pipe is being created,

Note: the pipe is opened, closed and written to in the same way we do with a file.

See example. [mkfifo.cpp](#)

- Look at the name with an `ls -l` command.
- Look at what happens if I try and create the pipe again.

- In the example we will try combinations of opening different ends of the pipe with different rules.
- We will play with delayed openings to see the effect on pipes. Some versions of UNIX get upset if you try to write to a pipe that no one has open for reading.
- We will also allow the process to write with no one reading and see if the data persists. Note that the file created for the pipe does not hold data.

Lab9

Write a program that will schedule 3 clones. Each clone will communicate to the parent process over a pipe. (ie. There are 3 pipes.) Each process will write "Process n.t" (where n is the number of the process 1, 2, or 3 and t is the number of messages that the process n has sent) to the parent 10 times at time intervals of $n^2 + 1$. The parent should display these messages as soon as they are received. Make sure that only one message is received at a time. We will talk about how to do this.

